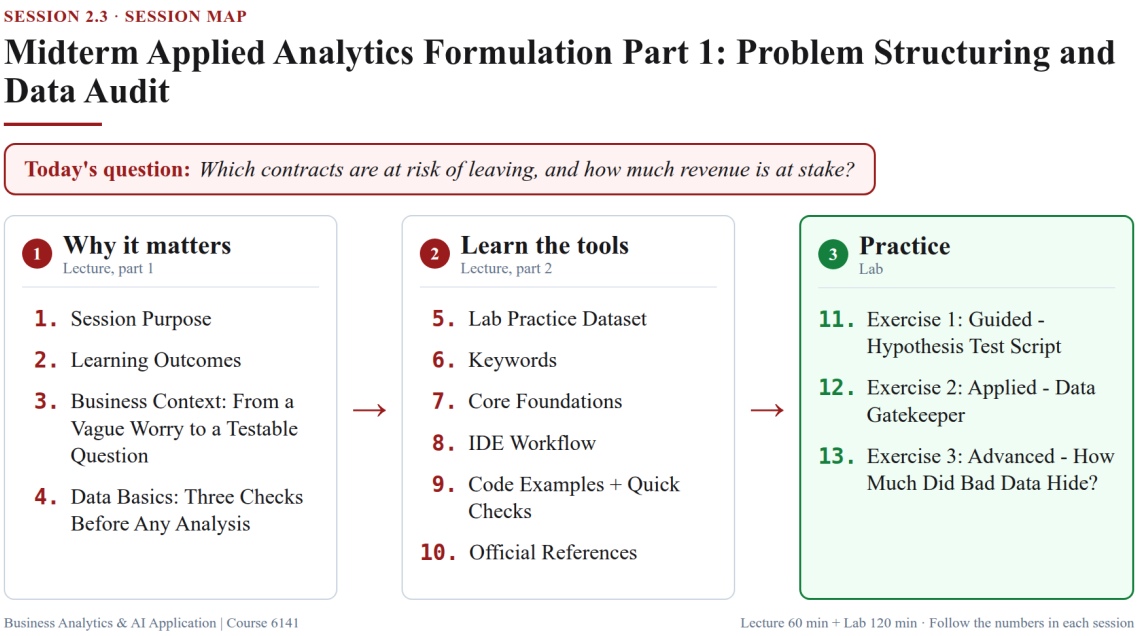
Session 2.3: Midterm Applied Analytics Formulation Part 1: Problem Structuring and Data Audit

Course 6141: Business Analytics & AI Application | Reading for Week 7



Schedule: Week 7 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

1. Session Purpose

- An IT (Information Technology) services company holds 15 enterprise client contracts worth $10.7 million a year
- The account director has a vague worry: “some clients are unhappy and may leave”, but a worry is not something data can test
- The account director asks: which contracts are at risk of leaving, and how much revenue is at stake? Today, you turn that worry into a testable question, check that the data can be trusted, and measure the starting point. This is step 1 of your midterm project

2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. “What exactly should we test?”: turn a vague business problem into a testable hypothesis (a clear statement that data can support or prove wrong)

2. "Can we trust this data?": audit a dataset for missing values (`.isnull()`), duplicates (`.duplicated()`), and impossible values, such as a percentage above 100
3. "How do we stop bad data automatically?": write `assert` checks (rules the data must pass) and handle an `AssertionError` with `try` / `except`

3. Business Context: From a Vague Worry to a Testable Question

- Managers rarely hand you a clean question. A useful analysis moves through four steps:

Step	Example
Vague worry	"Some clients are unhappy and may leave."
Business question	"Is poor service linked to clients at risk of leaving?"
Testable hypothesis	"Accounts below 95% SLA compliance have more billing disputes and a higher retention risk than other accounts."
Measure today (baseline)	"Revenue in High-risk accounts today: $930,000."

- SLA (Service Level Agreement): the service promise in a contract, such as "the system is available 99% of the time". SLA compliance is the share of that promise actually delivered.
- Baseline: today's value of the measure you care about. Without it, nobody can prove later that an action improved anything.
- **Midterm Project, Step 1 of 2: your Question Card.** Choose a business problem with a dataset you can audit (for example, any dataset from Sessions 1.3 to 2.2), and fill in five lines:
    1. Vague worry: what the manager is concerned about
    2. Business question: what decision the answer supports
    3. Hypothesis: one sentence the data can support or prove wrong
    4. Data and audit: the file you use, and the result of the three checks in section 4
    5. Baseline: one number that describes today
- Bring your completed Question Card and your audit script to Session 2.4, where you present your findings.

4. Data Basics: Three Checks Before Any Analysis

- Bad data produces confident but wrong answers ("garbage in, garbage out"). Run three checks before you analyze:

Check	Question	pandas
Completeness	Are any values missing?	`df.isnull().sum()`
Uniqueness	Does each ID appear only once?	`df["Account_ID"].duplicated().sum()`

Check	Question	pandas
Validity	Are all values possible?	`(df["SLA_Compliance_Pct"] > 100).sum()`

- `NaN` ("Not a Number") is how pandas shows an empty cell. `.isnull()` marks each `NaN` as `True`.
- A rule for validity comes from business logic: a percentage must be between 0 and 100, and a contract value must be above 0.

5. Lab Practice Dataset

- Two versions of the same 15 client contracts (company names are fictional):
  - Clean file: `data/session_02_03_dataset.csv`, 15 rows that pass every check. Use it for analysis.
  - Raw file: `data/session_02_03_raw.csv`, 16 rows as they arrived from the billing system, with a few problems for you to find. Use it for auditing.
- Columns (the same in both files):
  - `Account_ID`: Unique client account code (e.g., `ACC-301`).
  - `Enterprise_Client`: Client company name (Titan Tech, Beacon Health, Vanguard Retail, and others). One company can hold several accounts.
  - `Contract_Value_USD`: Yearly contract value in U.S. Dollars.
  - `SLA_Compliance_Pct`: Share of the service promise delivered, in percent.
  - `Billing_Discrepancy_Count`: Number of billing disputes (invoices the client challenged) during the contract.
  - `Client_Retention_Risk`: The account team's rating of how likely the client is to leave: `Low`, `Medium`, or `High`.

6. Keywords (Core Programming Concepts)

- `Hypothesis`:
  - What: one clear statement that data can support or prove wrong
  - Why: turns a vague worry into a test with a yes or no answer
- `Exploratory Data Analysis (EDA)`:
  - What: a first look at the data with counts, averages, and charts before any formal test
  - Why: shows what the data can answer, and where it looks strange
- `Data Audit`:
  - What: the three checks: completeness, uniqueness, and validity
  - Why: one bad row can change a total by millions
- `Baseline`:
  - What: today's value of the measure you care about

  - Why: the starting point for proving any later improvement

- `assert`:
  - What: a line that stops the script with an `AssertionError` when a rule is broken, such as `assert (df["Contract_Value_USD"] > 0).all()`
  - Why: bad data is caught at the door, before it reaches a report

- `try` / `except`:
  - What: runs code in `try`; if a chosen error happens, the code in `except` runs instead of stopping the script
  - Why: the script can report the problem clearly and keep going

7. Core Programming & Computational Foundations

- Counting `True` values:
  - A condition on a column gives one `True` / `False` per row
  - `.sum()` counts the `True` values, because Python treats `True` as 1 and `False` as 0
  - `df.isnull().sum().sum()`: the first `.sum()` counts per column, the second adds the columns together

- `.all()` and `.any()`:
  - `.all()` is `True` only if every value is `True`; `.any()` is `True` if at least one is
  - `assert (df["Contract_Value_USD"] > 0).all()` therefore means "every contract value is above zero"

- `assert` with a message:
  - `assert rule, "message"` shows your message in the `AssertionError`, so the reader knows which rule failed

- `try` / `except`:
  - Put the checks inside `try:`, and handle a failure under `except AssertionError as error:`
  - `error` holds the message, so you can print or save it

8. Development Workflow in Google Antigravity IDE

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S23_demo_1.py`), and check both files are in `data/`.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors.

9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input**, **2. Process**, and **3. Output**. Name each file with your own name and the session code, for example `Reacher_S23_demo_1.py` (pattern: `YourName_S23_demo_1.py`).
- *Micro-Example 1 (Audit the Raw File):* `YourName_S23_demo_1.py`

```python
# 1. Input: the raw file, as it arrived
import pandas as pd
raw = pd.read_csv("data/session_02_03_raw.csv")

# 2. Process: the three checks, as counts
missing_values = raw.isnull().sum().sum()
duplicate_ids = raw["Account_ID"].duplicated().sum()
sla_above_100 = (raw["SLA_Compliance_Pct"] > 100).sum()
value_not_positive = (raw["Contract_Value_USD"] <= 0).sum()

# 3. Output: one line per check
print("Rows:", len(raw))
print("Missing values:", missing_values)
print("Duplicate account IDs:", duplicate_ids)
print("SLA above 100%:", sla_above_100)
print("Contract value of 0 or less:", value_not_positive)
```

  - `.duplicated()` marks a row `True` when its value already appeared in an earlier row, so the first copy is not counted.
  - Result: 16 rows, and each check finds 1 problem.

- *Quick Check 1 (`.drop_duplicates()`):*
  - New tool: `raw.drop_duplicates()` returns the table without rows that exactly repeat an earlier row. Why: a copied row counts the same contract twice.
  - Task: add one line to part **3. Output**: `print("Rows after removing duplicates:", len(raw._____())))`
  - Check: the Terminal shows `Rows after removing duplicates: 15`.
- *Micro-Example 2 (Test the Hypothesis on the Clean File):* `YourName_S23_demo_2.py`

```python
# 1. Input: the clean file
import pandas as pd
df = pd.read_csv("data/session_02_03_dataset.csv")

# 2. Process: mark SLA breaches, then compare the two groups
df["SLA_Breach"] = df["SLA_Compliance_Pct"] < 95.0
comparison = df.groupby("SLA_Breach").agg(
    Accounts=("Account_ID", "count"),
    Avg_Disputes=("Billing_Discrepancy_Count", "mean"),
).round(2)
```

```
# 3. Output: breach (True) vs no breach (False)
print(comparison)
```

  - `SLA_Breach` is `True` for accounts below the 95% service promise.
  - Result: the 4 breached accounts average `4.75` billing disputes, against `0.64` for the other 11. The first half of the hypothesis is supported.

- *Quick Check 2 (`pd.crosstab()`):*

  - New tool: `pd.crosstab(column_a, column_b)` counts the rows for every pair of values, such as breach yes or no against risk Low, Medium, or High. Why: tests the second half of the hypothesis in one table.
  - Task: add one line to part **3. Output**: `print(pd._____(df["SLA_Breach"], df["Client_Retention_Risk"]))`
  - Check: in the `True` row, `High` is `3` and `Medium` is `1`: 3 of the 4 breached accounts are rated High risk, and no non-breached account is.

- *Micro-Example 3 (Measure the Baseline):* `YourName_S23_demo_3.py`

```
# 1. Input: the clean file
import pandas as pd
df = pd.read_csv("data/session_02_03_dataset.csv")

# 2. Process: contract value by risk rating, and the High-risk share
value_by_risk = df.groupby("Client_Retention_Risk")["Contract_Value_USD"].sum()
high_risk_value = value_by_risk["High"]
high_risk_share = high_risk_value / df["Contract_Value_USD"].sum() * 100

# 3. Output: today's baseline
print(value_by_risk)
print("High-risk share of revenue (%):", round(high_risk_share, 1))
```

  - `df.groupby(...)["Contract_Value_USD"].sum()` sums one column per group. `value_by_risk["High"]` reads the value for the `High` group.
  - Result: `930000.0` of revenue sits in High-risk accounts, `8.7`% of the total.

- *Quick Check 3 (f-string number format):*

  - New tool: `f"${value:,.0f}"` writes a number with thousands separators and no decimals, such as `$930,000`. Why: managers read money faster in this form.
  - Task: add one line to part **3. Output**: `print(f"Revenue at High risk: ${high_risk_value:___}")`
  - Check: the Terminal shows `Revenue at High risk: $930,000`.

10. Official Python Documentation & References

- Python Tutorial: Errors and Exceptions (`try` / `except`).
- Python Reference: The `assert` Statement.

- Pandas User Guide: Working with Missing Data.

11. Exercise 1: Guided Practice - Hypothesis Test Script

- *Step 1: Open Script:* In Google Antigravity IDE, create a new script named `YourName_S23_hypothesis.py`, replacing `YourName` with your own name (for example, `Reacher_S23_hypothesis.py`).
- *Step 2: Test Two Hypotheses and Measure the Baseline:* Type the following code:

```
# 1. Input: the clean file
import pandas as pd
df = pd.read_csv("data/session_02_03_dataset.csv")

# 2. Process: two tests and one baseline
# Hypothesis 1: higher-risk accounts have more billing disputes
disputes_by_risk = (
    df.groupby("Client_Retention_Risk")["Billing_Discrepancy_Count"]
    .mean().round(2)
)
# Hypothesis 2: lower SLA compliance goes with more disputes
sla_vs_disputes = df["SLA_Compliance_Pct"].corr(
    df["Billing_Discrepancy_Count"]
)
# Baseline: revenue in High-risk accounts today
high_risk_value = df.loc[
    df["Client_Retention_Risk"] == "High", "Contract_Value_USD"
].sum()

# 3. Output: a short report
print("Hypothesis 1: average disputes by risk rating\n", disputes_by_risk)
print(f"Hypothesis 2: correlation of SLA and disputes: {sla_vs_disputes:.3f}")
print(f"Baseline: revenue at High risk: ${high_risk_value:,.0f}")
```

  - `df.loc[rows, "Column_Name"]` selects the rows and one column at the same time.
  - `{sla_vs_disputes:.3f}` shows the number with 3 decimals.

- *Step 3: Interpret:* Write two comment lines at the end of the script: does each result support its hypothesis?
- *Step 4: Execute in Terminal:* Run `python YourName_S23_hypothesis.py` in Panel `[4]` and confirm return code `0`.
- *Success Check:* average disputes are `5.33` for High, `2.0` for Medium, and `0.25` for Low; the correlation is `-0.982` (lower SLA goes with more disputes); the baseline is `$930,000`. Both hypotheses are supported.

12. Exercise 2: Applied Practice - Data Gatekeeper

- *Coding Challenge:* In `YourName_S23_practice.py`, write checks that stop bad data, then clean the raw file so it passes.
- *Task 1 (Write the Rules):* Load `data/session_02_03_raw.csv`. Write three `assert` lines with messages:
  - `assert raw["SLA_Compliance_Pct"].between(0, 100).all(), "SLA must be between 0 and 100"`
  - `assert (raw["Contract_Value_USD"] > 0).all(), "Contract value must be above 0"`
  - `assert not raw["Account_ID"].duplicated().any(), "Account IDs must be unique"`
  - `.between(0, 100)` is `True` when a value is from 0 to 100; a missing value counts as `False`.
- *Task 2 (Read the AssertionError & AI Diagnosis):* Run the script. It stops at the first rule with `AssertionError: SLA must be between 0 and 100`. Capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and ask which rows break the rule.
- *Task 3 (Clean, Re-check, Save):* Put the three `assert` lines inside `try:` and print the message under `except AssertionError as error:`. Then build `clean` from `raw`: remove duplicate rows with `.drop_duplicates()`, keep only rows where the SLA is between 0 and 100, and keep only rows where the contract value is above 0. Run the three checks again on `clean`, and save it to `data/contracts_clean.csv` with `index=False`.
- *Success Check:* the first run prints `SLA must be between 0 and 100`; `clean` has `12` rows and passes all three checks; the script ends with return code `0`.

13. Exercise 3: Advanced Practice - How Much Did Bad Data Hide?

- *Goal:* Show the account director, in numbers, how the raw file's problems distort the risk report, and list exactly which accounts the data team must fix.
- *What you need:*
  - Files: `data/session_02_03_raw.csv` (with problems) and `data/session_02_03_dataset.csv` (the corrected version of the same contracts).
  - Script name: `YourName_S23_challenge.py`.
  - Tools you already know: the three checks, `groupby()`, `.sum()`, Boolean masks with `|`, `.loc[]`, `pd.DataFrame()`, and f-strings.
- *Requirements (follow them in order):*
  1. Write a function `audit(table)` that returns a dictionary with four counts: `"missing"`, `"duplicate_ids"`, `"sla_out_of_range"` (SLA above 100), and `"value_not_positive"` (contract value of 0 or less).
  2. Call `audit()` on both files and print both dictionaries.
  3. Make a list of every `Account_ID` in the raw file that breaks at least one rule. Each account should appear only once, sorted from smallest to largest.
  4. For each file, sum `Contract_Value_USD` by `Client_Retention_Risk`. Put the two results side by side in one DataFrame with the columns `Raw_USD` and `Correct_USD`, and add `Difference_USD` = `Raw_USD` - `Correct_USD`.

5. Print the table and one sentence that states the biggest distortion in plain words, using an f-string with thousands separators.

- *New tools for this exercise:*
  - Defining a function: `def audit(table):` followed by indented lines, ending with `return {...}`. Calling `audit(raw)` runs those lines on `raw` and gives back the dictionary.
  - `|` means "or" for masks: `(rule_1) | (rule_2)` is `True` when at least one rule is `True`. `.isnull()` on one column marks its missing values.
  - Unique, sorted values: `sorted(set(raw.loc[mask, "Account_ID"]))`. `set()` keeps each value once; `sorted()` puts them in order.
  - Side-by-side columns: `pd.DataFrame({"Raw_USD": raw_sums, "Correct_USD": correct_sums})`, where both sums come from `groupby()`, lines up the values by risk rating.
- *Hints:*
  - For requirement 3, build one mask that is `True` for a duplicate, a missing value, an SLA above 100, or a value of 0 or less. Use `raw.duplicated("Account_ID", keep=False)` so that both copies of a duplicated account are marked.
  - Print each intermediate result before you combine them.
- *Self-Check (expected results):*
  - Raw file: `{"missing": 1, "duplicate_ids": 1, "sla_out_of_range": 1, "value_not_positive": 1}`; clean file: all four counts are `0`.
  - Accounts to fix: `['ACC-304', 'ACC-306', 'ACC-311', 'ACC-313']`.
  - Differences: `High -620000.0`, `Low 1200000.0`, `Medium 0.0`.
  - Business meaning: one negative typo (`ACC-313`) made High-risk revenue look like `$310,000` instead of `$930,000`, hiding two-thirds of the real risk, while one duplicated row added `$1,200,000` to Low risk. The data must be fixed before anyone acts on the risk report.